

PROBLEMAS DE ESTRUCTURA DE COMPUTADORES

MEMORIA CACHE

Problema 1

Un computador dispone de un sistema de memoria constituido por una memoria principal de 64 Kbytes y una memoria cache de 2 Kbytes, con emplazamiento asociativo por conjuntos de 4 vías, política de reemplazo FIFO, post-escritura y sin asignación en escritura. El tamaño del bloque es de 256 bytes. Se pide:

- Interpretación de los bits de la dirección física del sistema de memoria para Mp y para Mc.
- Esquema de correspondencia entre Mp y Mc.
- Si en un determinado instante el conjunto 0 contiene las etiquetas, están expresadas en decimal, (10, 8, 9, 11) y el conjunto 1 las etiquetas (8, 9, 10, 11) ¿Qué bloques de Mp están cargados en Mc?

Solución

Apartado a)

Tamaño Mp = 64K = 2^{16} Bytes $\rightarrow$ 16 bits de dirección

Tamaño Bloque = 256 Bytes = 2^8 Bytes $\rightarrow$ 8 bits para el Byte del bloque

Nº de Bloques en MP = $64K / 256B = 2^{16} / 2^8 = 2^8 = 256$ bloques en Mp

Cache con emplazamiento asociativo por conjuntos de 4 vías y tamaño 2KB= 2^{11} bytes

Nº de Bloques en Cache = $2K / 256B = 2^{11} / 2^8 = 2^3 = 8$ bloques en cache

Nº de conjuntos = 8 bloques / 4 bloques por conj= 2 $\rightarrow$ 1 bit de conjunto

Dirección vista por la MP:

8	8
B _{MP}	Byte del bloque

Dirección vista por la Cache:

7	1	8
Etiqueta	Conj	Byte del bloque

Apartado b) Nº de bloques en Mp = $64K/256 = 2^{16}/2^8 = 2^8 = 256$ bloques

Conjunto 0 $\leftarrow$ B0, B2, B4, ..., B254

Conjunto 1 $\leftarrow$ B1, B3, B5, ..., B255

Apartado c)

Cache		
Etiqueta (decimal)		
10		Conj0
8		
9		
11		
8		Conj1
9		
10		
11		

dirección del contenido de cada Bloque que hay en cache

Conjunto 0

Etiqueta	conj	Byte del bloq.		Bloque de Mp (en decimal)
000 1010	0	XXXX XXXX	→	0001 0100 B _{MP} 20
000 1000	0	XXXX XXXX	→	0001 0000 B _{MP} 16
000 1001	0	XXXX XXXX	→	0001 0010 B _{MP} 18
000 1011	0	XXXX XXXX	→	0001 0110 B _{MP} 22

Conjunto 1

Etiqueta	conj	Byte del bloq.		
000 1000	1	XXXX XXXX	→	0001 0001 B _{MP} 17
000 1001	1	XXXX XXXX	→	0001 0011 B _{MP} 19
000 1010	1	XXXX XXXX	→	0001 0101 B _{MP} 21
000 1011	1	XXXX XXXX	→	0001 0111 B _{MP} 23

Problema 2

Un computador dispone de un sistema de memoria constituido por una memoria principal de 64 KB y una memoria cache de 256 bytes con emplazamiento directo, post-escritura y sin asignación en escritura. El tamaño del bloque es de 64 Bytes.

- Indicar el formato de la dirección para MP y para la MC
- En un momento dado los valores de las etiquetas que hay en la cache (en hexadecimal) son los indicados en la tabla. Expresar en hexadecimal el bloque de MP Y el rango de direcciones de MP que están ubicadas en cada bloque de la memoria cache.

Etiqueta	Bloque
18	0
25	1
D3	2
51	3

- c) Supongamos que un programa realiza varios accesos de lectura a las siguientes direcciones de memoria (están expresadas en hexadecimal): D380, 6010, D3F0, 6018, 1830. Calcular el número de fallos que se producen e indicar cómo se queda el contenido de la memoria cache después de acceder a la última dirección.

Solución

Apartado a)

Tamaño Mp = 64K = 2^{16} Bytes $\rightarrow$ 16 bits de dirección

Tamaño Bloque = 64 Bytes = 2^6 bytes $\rightarrow$ 6 bits para el Byte del bloque

Nº de Bloques en MP = $64K / 64B = 2^{16} / 2^6 = 2^{10} = 1024$ bloques en Mp

Cache con emplazamiento directo y tamaño 256B = 2^8 bytes

Nº de Bloques en Cache = $256B / 64B = 2^8 / 2^6 = 2^2 = 4$ bloques en cache

Dirección vista por la MP:

10	6
B_{MP}	Byte del bloque

Dirección vista por la Cache:

8	2	6
Etiqueta	B_C	Byte del bloque

Apartado b)

B_{MP}

Etiqueta	B_C	Byte del bloq	Rango de dir.	Bloque de Mp (en Hex)
0001 1000	00	00 0000 – 11 1111	$\rightarrow$ 1800- 183F	00 0110 0000 $\rightarrow$ $B_{MP}060$
0010 0101	01	00 0000 – 11 1111	$\rightarrow$ 2540- 257F	00 1001 0101 $\rightarrow$ $B_{MP}095$
1101 0011	10	00 0000 – 11 1111	$\rightarrow$ D380- D3BF	11 0100 1110 $\rightarrow$ $B_{MP}34E$
0101 0001	11	00 0000 – 11 1111	$\rightarrow$ 51C0- 51FF	01 0100 0111 $\rightarrow$ $B_{MP}147$

Cache

Etiqueta
(Hex)

18	$B_{MP} 060$ 1800- 183F	B_{C0}
25	$B_{MP} 095$ 2540- 257F	B_{C1}
D3	$B_{MP} 34E$ D380- D3BF	B_{C2}
51	$B_{MP} 147$ 51C0- 51FF	B_{C3}

Apartado c)

B_{MP}

Etiqu Bc Byte del bloq

- (1) D380 → **D3 10** 00 0000 ACIERTO
(2) 6010 → **60 00** 01 0000, FALLO, reemplazamos el Bc0 por **01 1000 0000** → **B_{MP}108**
(3) D3F0 → **D3 11** 11 0000, FALLO, reemplazamos el Bc3 por **11 0100 1111** → **B_{MP}3F4**
(4) 6018 → **60 00** 01 1000, ACIERTO
(5) 1830 → **18 00** 11 0000, FALLO, reemplazamos el Bc0 por **00 0110 0000** → **B_{MP}060**

Cache

Etiqueta
(Hex)

18	B_{MP} 060	Bc0
25	B_{MP} 095	Bc1
D3	B_{MP} 34E	Bc2
D3	B_{MP} 3F4	Bc3

Problema 3

Una jerarquía de memoria está compuesta por una memoria principal de 16 Mbytes y una cache de 16 Kbytes con emplazamiento directo, post-escritura y con asignación en escritura. El tamaño de bloque es de 256 bytes. Sobre esta jerarquía de memoria se ejecuta un millón de accesos de lectura, obteniéndose una tasa de aciertos de 0,995. Se supone que el tiempo de acceso a la memoria principal es de 20 ns, el tiempo de acceso a la cache es de 1 ns y la penalización por fallo es de 400 ns. Responde a las siguientes cuestiones (0,5 puntos cada apartado):

Nota: Todos los datos numéricos del problema están en decimal. Las respuestas numéricas pueden darse en el sistema decimal o hexadecimal, a elección del estudiante.

- a) Muestra el formato de la dirección para la Mp y la cache.
b) Si el bloque n° 87 (decimal) de la Mp tiene que llevarse a la cache ¿qué n° de bloque de cache se le asigna?
c) ¿A cuántos bloques de Mp le corresponde el mismo bloque en la cache?
d) Indica el n° del primer y el último bloque de Mp a los que les corresponde alojarse en el bloque 22 (decimal) de la cache.
e) ¿Cuál es el tiempo medio de acceso a memoria?
f) ¿Cuánto se tarda en ejecutar la secuencia de accesos a memoria descrita?
g) ¿Qué ganancia de velocidad se obtiene respecto del mismo sistema, pero sin la cache?

Solución:

Apartado a)

Tamaño Mp: 16 Mbytes = 2^{24} bytes → 24 de dirección

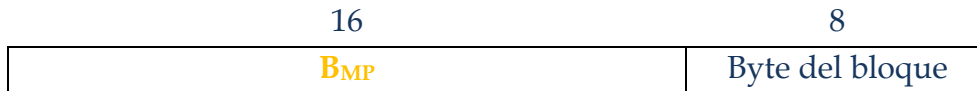
Tamaño Bloque: 256 bytes = 2^8 → 8 bits para el Byte del bloque

Nº de Bloques en MP = $16\text{MB} / 256\text{B} = 2^{24} / 2^8 = 2^{16}$ bloques en Mp

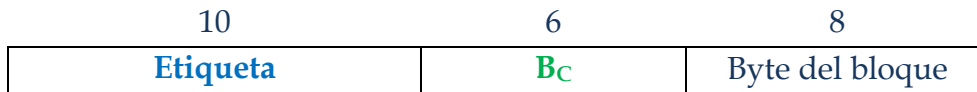
Cache con emplazamiento directo y tamaño 16KB= 2^{14} bytes

Nº de Bloques en Cache = $16\text{KB} / 256\text{B} = 2^{14} / 2^8 = 2^6 = 64$ bloques en cache

Dirección vista por la MP:



Dirección vista por la Cache:



Apartado b)

Bloque de Mp: 87 = (0000 0000 0101 0111)₂ Los 6 bits menos significativos determinan el bloque de cache → bloque de cache 0x17 = 23 en decimal.

Apartado c)

Todos los bloques de Mp que se mapean sobre el mismo bloque de cache se caracterizan por tener idénticos los 6 bits menos significativos. Los restantes 10 bits (que corresponden al campo de TAG) pueden tomar cualquier valor, desde 00 0000 0000 hasta 11 1111 1111. Luego hay $2^{10} = 1024$ bloques de Mp que se mapean sobre un mismo bloque de cache.

Apartado d)

Los bloques de Mp que alojan sobre el bloque 22 (decimal) de la cache son de la forma:

XXXX XXXX XX01 0110

Luego el primero es 0000 0000 0001 0110 = 0x0016 y el último es 1111 1111 1101 0110 = 0xFFD6.

Apartado e)

$TMAM = T_{acierto} + TasaFallos \times T_{penalty} = 1 + (1 - 0.995) \cdot 400 = 3 \text{ ns}$

$TasaFallos = (1 - TasaAcertos)$

Tacerto = tiempo de acceso a la cache

Estamos suponiendo sistema de memoria de lectura no directa, es decir en caso de fallo primero se lleva el bloque a la cache y luego se vuelve a acceder a la cache para llevar el dato a la CPU. Si no sería así: $TMAM = T_{acerto} + TasaFallos \times T_{penalty}$

Apartado f)

$T = (\text{no. referencias}) \times TMAM = 10^6 \times 3 \times 10^{-9} = 3 \text{ ms}$

Apartado g)

$T \text{ sin cache} = (\text{no. referencias}) \times 20 \text{ ns} = 10^6 \times 20 \times 10^{-9} = 20 \text{ ms}$

Ganancia de velocidad = $20 / 3 = 6,66$ veces más rápido con la cache.

Problema 4

Sea un computador con un procesador ARMv4. EL procesador cuenta con una memoria cache para datos y otra cache para instrucciones ambas de 256 bytes, emplazamiento directo, post-escritura y asignación en escritura. El tamaño del bloque es de 64 bytes.

En dicho sistema se quiere ejecutar el siguiente programa.

```
.equ N, 32

.data
A: .word N valores enteros separados por comas
B: .word N valores enteros separados por comas

.bss
C: .space N*4

.text
start: ldr r0, =A
      ldr r1, =B
      ldr r2, =C
      mov r4, #N
L1:   ldr r5, [r0]
      ldr r6, [r1]
      add r5, r5, r6
      mov r5, r5, lsl #2
      str r5, [r2]
      add r0, r0, #4
      add r1, r1, #4
      add r2, r2, #4
      sub r4, r4, #1
      cmp r4, #0
      bne L1
end: b .
```

NOTA: Este programa sería equivalente al siguiente código C

```
for( i=0; i < N; i++ )
    C[i] = (A[i] + B[i]) * 4;
```

- Con direcciones de 32 bits, indicar el formato de la dirección para MP y para la MC.
- El programa se enlaza ubicando la sección .data a partir de la dirección 0x0C000000, con la sección .bss colocada justo después de la sección .data y la sección .text a continuación de la sección .bss. Obtenga los rangos de direcciones que ocupan el array A, el array B, el array C y las instrucciones indicando el rango de direcciones y el bloque de cache que le corresponde.
- Determina el número de fallos que se producen al ejecutar el programa completo. Indica como quedan las dos caches al finalizar el programa. No hace falta poner la etiqueta, solo el contenido del bloque.
- ¿Mejoraría el número de fallos si se cambiara la cache de datos por una igual, pero **sin asignación en escritura**? **Razona la respuesta**
- ¿Mejoraría el número de fallos si se cambiara la cache de datos por una del mismo tamaño, pero con emplazamiento asociativo por conjuntos de 2 vías, política de reemplazo FIFO, post-escritura y asignación en escritura? **Razona la respuesta**

Solución:

Apartado a)

32 de dirección → tamaño Mp $2^{32} = 4\text{GB}$

Tamaño Bloque: 64 bytes = 2^6 → 6 bits para el Byte del bloque

Cache con emplazamiento directo y tamaño $256\text{B} = 2^8$

Nº de Bloques en Cache = $256\text{B} / 64\text{B} = 2^8 / 2^6 = 2^2 = 4$ bloques en cache

Dirección vista por la MP:

26	6
B_{MP}	Byte del bloque

Dirección vista por la Cache:

24	2	6
Etiqueta	B_C	Byte del bloque

Apartado b)

El código son 16 palabras = 64 bytes, es decir, cabe en un bloque. La tabla de literales requiere otro bloque (solo usa las 3 primeras palabras)

Cada array ocupa 32 palabras = 128 bytes, con lo que cada array ocupa dos bloques.

$(128)_{10} = (1000\ 0000)_2 = 0x80$

Dirección de comienzo del array A 0x0c000000
en los Bc 0 y 1 de la cache de datos

Etiqueta **Bc** **Bytes del bloque**
0000 1100 ... 0000 0000 0000 Como ocupa dos bloques se coloca

Dirección de comienzo del array B 0x0c000080
en los Bc 2 y 3 de la cache de datos

0000 1100 ... 0000 1000 0000 Como ocupa dos bloques se coloca

Dirección de comienzo del array C 0x0c000100
en los Bc 0 y 1 de la cache de datos

0000 1100 ... 0001 0000 0000 Como ocupa dos bloques se coloca

Dirección de comienzo del programa 0x0c000180

0000 1100 ... 0001 1000 0000 B2 de la cache de instrucciones

Dir. de comienzo de la tabla de literales 0x0c0001C0

0000 1100 ... 0001 1100 0000 B3 de la cache de instrucciones

	rango	Bloque de cache
Array A	0x0c000000 - 0x0c00007F	Bc0 y Bc1 (de datos)
Array B	0x0c000080 - 0x0c0000FF	Bc2 y Bc3 (de datos)
Array C	0x0c000100 - 0x0c00017F	Bc0 y Bc1 (de datos)
Instrucciones	0x0c000180 - 0x0c0001BF 0x0c0001C0 - 0x0c0001CB	Bc2 y Bc3 (de instrucciones) (Bc2 el programa) (Bc3 tabla de literales)

Apartado c) Las caches tras la primera vuelta del bucle contienen lo siguiente:

Cache de datos	
Etiqueta	
	A(0)-A(15) C(0)-C(15) B_C 0
	A(16)-A(31) C(16)-C(31) B_C 1
	B(0)-B(15) B_C 2
	B(16)-B(31) B_C 3

Cache de Instrucciones	
Etiqueta	
	B_C 0
	B_C 1
	Prog B_C 2
	Tabla literales B_C 3

Instrucciones: Todas las instrucciones tienen accesos a Fetchs:

2 fallos en la primera instrucción, uno en la etapa fetch (se lleva a cache el bloque que tiene todo el código y se coloca en el Bc2) y otro en la etapa Mem (se lleva a cache el bloque que tiene la table de literales y se coloca en el Bc3)
0 falos el resto de las instrucciones.

Datos:

En cada vuelta hay 2 lecturas (ldr) y 1 escritura (str)

i=0 Lectura de A(0) produce fallo, se coloca en Bc 0 y se reemplaza al escribir C(0) → **2 fallos**

Lectura de B(0) produce fallo, se coloca en Bc 2 → **1 fallo**

i=1-15 Se produce fallo de lect A y fallo de esc C porque compiten por el Bc → **2 fallos por vuelta**

Lectura de B(i) NO produce fallo

i=16 Fallo de lect A(16), se coloca en Bc 1 y se reemplaza al escribir C(16) → **2 fallos**

Fallo de lect B(16), se coloca en Bc 3 → **1 fallo**

i=17-31 Se produce fallo de lect A y fallo de esc C porque compiten por el Bc → **2 fallos por vuelta**

Lectura de B(i) NO produce fallo

$$\text{Fallos en la cache de datos} = 3 + 2 \times 15 + 3 + 2 \times 15 = 66$$

$$(i=0) \quad (i=1-15) \quad (i=16) \quad (i=17-31)$$

Apartado d) ¿Mejoraría el número de fallos si se cambiara la cache de datos por una igual, pero sin asignación en escritura? Razona la respuesta

Solo afecta a la cache de datos. En este caso el array A y C no compiten por Bc porque el array C nunca se lleva a Cache.

La escritura de C siempre produce fallo → 32 escrituras **32 fallos**

Lectura de A(0) produce fallo, se coloca en Bc 0 → **1 fallo**

Lectura de A(16) produce fallo, se coloca en Bc 1 → **1 fallo**

Lectura de B(0) produce fallo, se coloca en Bc 2 → **1 fallo**

Lectura de B(16) produce fallo, se coloca en Bc 3 → **1 fallo**

Fallos en la cache de datos = 36

Apartado e) ¿Mejoraría el número de fallos si se cambiara la cache de datos por una del mismo tamaño, con emplazamiento asociativo por conjuntos de 2 vías, política de reemplazo FIFO, post-escritura y asignación en escritura? Razona la respuesta

Solo afecta a la cache de datos.

Dirección vista por la Cache:

25	1	6
Etiqueta	Conj	Byte del bloque

	Etiqueta	Conj	Bytes del bloque	
Dirección de comienzo del array A 0x0c000000	0000 1100 ...	0000 0 0	00 0000	Como ocupa dos bloques, el primer bloque se coloca en el conjunto 0 y el segundo bloque en el conjunto 1 de la cache de datos
Dirección de comienzo del array B 0x0c000080	0000 1100 ...	0000 1 0	00 0000	Como ocupa dos bloques, el primer bloque se coloca en el conjunto 0 y el segundo bloque en el conjunto 1 de la cache de datos
Dirección de comienzo del array C 0x0c000100	0000 1100 ...	0001 0 0	00 0000	Como ocupa dos bloques, el primer bloque se coloca en el conjunto 0 y el segundo bloque en el conjunto 1 de la cache de datos

En este caso a los arrays A, B y C les corresponde el conjunto 0, y no caben los tres porque cada conjunto solo tiene dos bloques, por lo que van a producirse reemplazamientos.

i=0 Lectura de A(0) produce fallo, se coloca en Bc 0 del Conjunto 0 → **1 fallo**

Lectura de B(0) produce fallo, se coloca en Bc 1 del Conjunto 0 → **1 fallo**

Escritura de C(0) produce fallo, se reemplaza A (política FIFO) coloca en Bc 0 del Conjunto 0 → **1 fallo**

Cache de datos asociativa cuando acaba la iteración i=0

Etiqueta

	A(0)-A(15) C(0)-C(15)	Conj 0
	B(0)-B(15)	
		Conj 1

i=1

Lectura de A(1) produce fallo porque A se reemplazó. Se coloca en Bc 1 del Conjunto 0, se reemplaza B (política FIFO)

→ **1 fallo**

Lectura de B(1) produce fallo porque B se acaba de reemplazar. Se coloca en Bc 0 del Conjunto 0, se reemplaza C (política FIFO) → **1 fallo**

Escritura de C(0) produce fallo, porque C se acaba de reemplazar. Se coloca en Bc 1 del Conjunto 0, se reemplaza A (política FIFO) → **1 fallo**

Cache de datos asociativa cuando acaba la iteración i=1

Etiqueta

	C(0)-C(15) B(0)-B(15)	Conj 0
	B(0)-B(15) A(0)-A(15) C(0)-C(15)	
		Conj 1

Como vemos en cada vuelta se producen 3 fallos, por lo tanto aumenta el nº de fallos.

Problema 5

Supongamos una jerarquía de memoria que consta de una Mp direccionable por palabras, donde cada palabra tiene una anchura de 32 bits, y una Mc de emplazamiento directo con 128 bloques, siendo cada bloque de 512 palabras. Además, sabemos que:

- Las direcciones de Mc tienen un campo de etiqueta de 8 bits.
- El tiempo de acceso a la Mc es 1 ciclo de reloj.
- La penalización por fallo es 130 ciclos de reloj.
- El tiempo de acceso a Mp es 10 ciclos de reloj.

Sobre esta jerarquía se ejecuta 10 veces seguidas una cadena de referencias entre las direcciones 0xF59600 y 0xF698C7, ambas inclusive, estando la cache inicialmente vacía.

- ¿Cuál es el tamaño de la Mp expresado en megabytes?
- Determina el número total de fallos producidos.
- Teniendo en cuenta la siguiente definición, determina la Tasa de Fallos en este problema.

$$Tasa\ de\ Fallos = \frac{Número\ total\ de\ fallos}{Número\ total\ de\ referencias\ a\ memoria}$$

- d) Determina el tiempo medio de acceso a memoria.
e) Calcula la mejora de velocidad que se obtiene al emplear esta jerarquía de memoria, en comparación con la alternativa de usar solamente la Mp.

Solución:

Apartado a)

Tamaño Bloque: 512 palabras = $2^9 \rightarrow$ 9 bits para la palabra del bloque

Cache con emplazamiento directo y 128 bloques = $2^7 \rightarrow$ 7 bits para el Bc

El enunciado dice que la etiqueta tiene 8 bits

Dirección vista por la MP:

15	9
B_{MP}	Palabra del bloque

Dirección vista por la Cache:

8	7	9
Etiqueta	B_C	Palabra del bloque

La dirección es de 24 bits

La Mp tiene 2^{24} palabras = 2^{26} bytes $\rightarrow$ el tamaño de la Mp es de 64 MB

Apartado b)

Tamaño de la Mc = 128 bloques

Cuántos bloques ocupan el número de accesos que se realizan (desde la direc 0xF59600 hasta la direc 0xF698C7, ambas incluidas):

0xF59600 = **1111 0101 1001 0110** 0000 0000 $\Rightarrow$ Bloque Mp = 0x7ACB

0xF698C7 = **1111 0110 1001 1000** 1100 0111 $\Rightarrow$ Bloque Mp = 0x7B4C

Tamaño = 7B4C – 7ACB + 1 = 0x82 = 130 bloques

Todos los bloques que ocupan estos accesos no caben en la Mc. Los dos últimos bloques se ubican en los mismos marcos de bloque (Bloques de cache) que los dos primeros bloques, luego en cada iteración se reemplazan.

1ª iteración: 130 fallos

Resto de iteraciones: 4 fallos por iteración $\rightarrow 4 \times 9 = 36$

Total Fallos = 166

Apartado c)

Tasa de fallos = N° fallos / N° total de accesos

N° total de accesos:

Cada vuelta se accede a 130 bloques, en el último bloque no se accede a todas las palabras solo de la 0 a la 199 (011000111), es decir 200 palabras.

$129 \times 512 + 200 = 66248$

Accesos totales= 10 x 66248= 662480

Tasa de fallos = 166 / 662480 = 0,00025 = 0,025 %

Apartado d)

TMAM = Tacierto + (1-H) Tpenal = 1 + 0,00025x130 = 1,0325 ciclos (en tr. de teoría)

Apartado e)

Mejora = Tiempo con solo Mp / Tiempo con jerarquía

T solo Mp = N° accesos x 10 = 6624800 ciclos

T con jeraquía = N° accesos x TMAM = 662480x1,0325 = 684011 ciclos

Mejora = 9,68 veces más rápido

Problema 6 (Examen septiembre 2020)

Se quiere ejecutar el siguiente programa basado en el ARM. El programa se enlaza ubicando la sección .data a partir de la dirección 0x00060 y la sección .text a continuación de la sección .data

Pseudocódigo en alto nivel

```
#define N 16
int A[N];
int B[N];

for (i=0; i < N; i++)
    B[i]= A[i]+1;
```

Código en ensamblador

```
.equ  N, 16

.data (A y B contiene 16 enteros)

A: .word -7,3,11,1,-5,9,0,1,0,0,-1,-3,15,6,7,1
B: .word 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0

.text
start: ldr R0, =A      (R0=Dir A)
       ldr R1, =B      (R1=dir B)
       mov R2, #0
       mov R3, #N
for:   cmp R2, R3
       bge fin
       ldr R4, [R0]    (R4=A[i])
       add R4, R4, #1
       str R4, [R1]    (B[i]=R4)
       add R0, R0, #4
       add R1, R1, #4
       add R2, R2, #1
       b for
fin:   b .
```

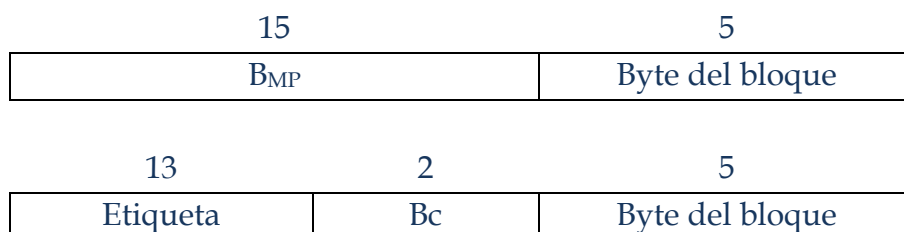
El sistema de memoria tiene una memoria principal de 1MB y una cache directa (de datos e instrucciones), con post-escritura y asignación en escritura. El tamaño de la cache es de 128Bytes y el tamaño de bloque de 32Bytes.

- Formato de la dirección de memoria principal y formato de la dirección de las cache
- Obtener la dirección en la que empieza el array A, el array B y el programa. Indicar qué bloques de memoria principal ocupan cada uno.
- De los bloques de memoria principal que ocupan los datos y el programa ¿A qué bloques les corresponde el mismo bloque de cache?

- d) Se sabe que el tiempo de acceso a la memoria principal es de 20 ns, el tiempo de acceso a la cache es de 1 ns, la tasa de aciertos de la cache es de 0,995 y la penalización por fallo es de 400 ns. Calcular el tiempo medio de acceso a memoria de este sistema de memoria.

Solución:

- a) Formato de la dirección de memoria principal y formato de la dirección de las caches
 Tamaño Bloque = 32Bytes = 2^5 bytes = 8 palabras
 Tamaño Mp = 1 MB = 2^{20} bytes
 Tamaño Cache = 128B = 2^7 bytes



- b) Array A: Empieza en la dirección: 0x00060 y ocupa 2 Bloques de MP: BMP 3 y BMP 4

Array B: Empieza en la dirección: 0x000A0 y ocupa 2 Bloques de MP: BMP 5 y BMP 6

Programa + TL= 12 instrucciones + 2 palabras de la TL: Empieza en la dirección: 0x000E0
 Ocupan cuatro Bloques de MP: BMP 7 , BMP 8

- c) A los BMP 3 y Bmp 7 les corresponde el Bc3

A los BMP 4 y Bmp 8 les corresponde el Bc 0

- d) $TMAM = T_{hit} + (1-H) T_{penalty} = 1 + (1-0.995) 400 = 3 \text{ ns}$

Problema 7 (Examen enero 2020)

Se quiere ejecutar el siguiente programa basado en el ARM. El programa se enlaza ubicando la sección .data a partir de la dirección 0x00020 y la sección .text a continuación de la sección .data.

Pseudocódigo en alto nivel

```
#define N 8
int A[N];
int B[N];

for (i=0; i < N; i++)
    B[i]= A[i]+1;
```

Código en ensamblador

```
.eq N, 8

.data (A y B contiene 8 enteros)

A: .word -7,3,11,1,-5,9,0,1
B: .word 0,0,0,0,0,0,0,0

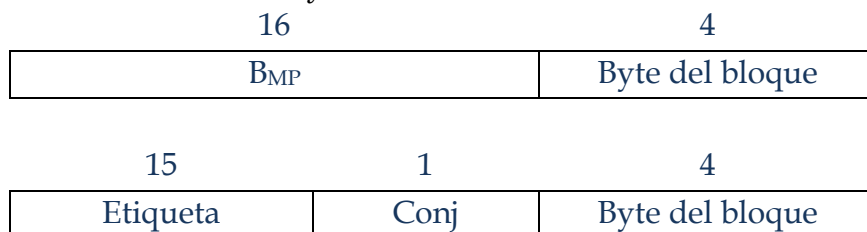
.text
start: ldr R0, =A      (R0= Dir A)
      ldr R1, =B      (R1= Dir B)
      mov R2, #0
      mov R3, #N
for:   cmp R2, R3
      bge fin
      ldr R4, [R0]    (R4=A[i])
      add R4, R4, #1
      str R4, [R1]    (B[i]=R4)
      add R0, R0, #4
      add R1, R1, #4
      add R2, R2, #1
      b for
fin:   b .
```

El sistema de memoria tiene una memoria principal de 1MB y una cache (de datos e instrucciones) asociativa por conjuntos de 2 vías con política de reemplazo FIFO, post-escritura y sin asignación en escritura. El tamaño de la cache es de 64Bytes y el tamaño de bloque de 16Bytes.

- Formato de la dirección de memoria principal y formato de la dirección de las cache
- Obtener la dirección en la que empieza el array A, el array B, el programa y la tabla de literales. Indicar qué bloques de memoria principal ocupan cada uno.
- Indicar el contenido de la cache después de ejecutar las vueltas $i=0, 1$ del bucle. Indicar el orden en el que se va llenando la cache.
- ¿Cuántos fallos se producen tras ejecutar esas dos vueltas ($i=0$ y 1)? Razona la respuesta.

Solución:

- Formato de la dirección de memoria principal y formato de la dirección de las caches
 Tamaño Bloque = 16 Bytes = 2^4 bytes = 4 palabras
 Tamaño Mp = 1 MB = 2^{20} bytes
 Tamaño Cache = 64B = 2^6 bytes



- Array A: Empieza en la dirección: 0x00020 y ocupa 2 Bloques de MP: BMP 2 y BMP 3

Array B: Empieza en la dirección: 0x00040 y ocupa 2 Bloques de MP: BMP 4 y BMP 5

Programa + TL= 16 instrucciones + 2 palabras de la TL: Empieza en la dirección: 0x00060
 Ocupan cuatro Bloques de MP: BMP 6 , BMP 7, BMP 8 y BMP 9

- Contenido de la cache después de ejecutar las vueltas $i=0, 1$ del bucle

En la vuelta i=0

Se lee A(0): BMP 3

Se escribe en B(0) pero como es sin asignación la matriz B nunca se trae

Orden	Etiqueta		
1º	0x0003	B_{MP} 6 (Inst-1)	Conj 0
5º	0x0004	B _{MP} 8 (Inst-2)	
4º	0x0001	A(0)-A(3)	
2º	0x0004	B _{MP} 9 (Inst y T.L)	Conj 1
3º	0x0003	B _{MP} 7 (Inst-1)	

i=0 → 6 fallos (4 fallos al traer los 4 bloques del programa + T.L, 1 fallo de lectura A(0) y fallo escritura B(0))

i=1 → Como todo lo que necesitamos está en la cache sólo hay 1 fallo de escritura B(1)